

Partie 2 : PL/PGSQL - LANGAGE DE PROCÉDURES SQL ET TRIGGERS

PL/pgSQL est un langage de procédures chargeable pour le système de bases de données **PostgreSQL™**.

Il est utilisé pour créer des fonctions standards et triggers. La fonction est créée en utilisant **CREATE OR REPLACE FUNCTION**.

L'ensemble des procédures stockées ainsi définies sont enregistrés dans un fichier texte .SQL.

On peut charger la procédure stockée (script sql) de 2 manières :

- 1) **createlang plpgsql mabase** // ajouter le langage de procédures PL/pgSQL dans la base
psql -d mabase -f /repertoire/fichier.sql // pour charger le script
- 2) pendant que psql tourne, on peut charger ou recharger de telles définitions de fonction avec
\i nom_fichier.sql

1. Structure de PL/pgSQL

PL/pgSQL est un langage structuré en blocs. Avec PL/pgSQL vous pouvez utiliser tous les types de données, opérateurs et fonctions du SQL.

Le texte complet de la définition d'une fonction doit être un bloc. Un bloc est défini comme :

```
[ <<label>> ]  
[ DECLARE  
  déclarations ]  
BEGIN  
  instructions  
END [ label ];
```

Chaque déclaration et chaque expression au sein du bloc est terminé par un point-virgule. Un bloc qui apparaît à l'intérieur d'un autre bloc doit avoir un point-virgule après END (voir l'exemple ci-dessus) ; néanmoins, le END final qui conclut le corps d'une fonction n'a pas besoin de point-virgule.

Tous les mots clés et identifiants peuvent être écrits en majuscules et minuscules mélangées. Les identifiants sont implicitement convertis en minuscule à moins d'être entourés de guillemets doubles.

Il y a deux types de commentaires dans PL/pgSQL. Un double tiret (--) débute une ligne de commentaire qui s'étend jusqu'à la fin de la ligne. Un /* débute un bloc de commentaire qui s'étend jusqu'à la prochaine occurrence de */. Les blocs de commentaires ne peuvent pas être imbriqués, mais les commentaires de lignes (double tiret)

peuvent être contenus dans un bloc de commentaire et un double tiret peut cacher les délimiteurs du bloc de commentaire /* et */. Chaque expression de la section expression d'un bloc peut être un sous-bloc. Les sous-blocs peuvent être utilisés pour des groupements logiques ou pour localiser des variables locales à un petit groupe d'instructions.

Les variables déclarées dans la section déclaration précédant un bloc sont initialisées à leur valeur par défaut chaque fois qu'on entre dans un bloc et pas seulement une fois à chaque appel de fonction. Par exemple :

```
CREATE FUNCTION une_fonction() RETURNS  
integer AS $$  
DECLARE  
  quantité integer := 30;  
BEGIN  
  RAISE NOTICE 'quantité vaut ici %', quantité;  
  -- quantité vaut ici 30  
  quantité := 50;  
  --  
  -- Crée un sous-bloc  
  --  
  DECLARE  
    quantité integer := 80;  
  BEGIN  
    RAISE NOTICE 'quantité vaut ici %',  
quantité; -- quantité vaut ici 80  
  END;  
  
  RAISE NOTICE 'quantité vaut ici %', quantité;  
  -- quantité vaut ici 50  
  
  RETURN quantité;  
END;  
$$ LANGUAGE plpgsql;
```

Il est important de ne pas confondre l'utilisation de **BEGIN/END** pour grouper les instructions dans PL/pgSQL avec les commandes de bases de données pour le contrôle des transactions. Les **BEGIN/END** de PL/pgSQL ne servent qu'au groupement ; ils ne débutent ni ne terminent une transaction. Les fonctions standards et les fonctions triggers sont toujours exécutées à l'intérieur d'une transaction établie par une requête extérieure -- ils ne peuvent pas être utilisés pour commencer ou valider une transaction car ils n'auraient pas de contexte pour s'exécuter. Néanmoins, un bloc contenant une clause EXCEPTION forme réellement une sous-transaction qui peut être annulée sans affecter la transaction externe.

1.1. Déclarations

Toutes les variables utilisées dans un bloc doivent être déclarées dans la section déclaration du bloc. La seule exception est que la variable de boucle d'une boucle FOR effectuant une itération sur des valeurs entières est automatiquement déclarée comme variable entière (type integer). Les variables PL/pgSQL peuvent être de n'importe quel type de données tels que integer, varchar et char. Voici quelques exemples de déclarations de variables :

```
id_utilisateur integer;
quantité numeric(5);
url varchar;
ma_ligne nom_table%ROWTYPE;
mon_champ nom_table.nom_colonne%TYPE;
une_ligne RECORD;
```

La syntaxe générale d'une déclaration de variable est :

```
nom [ CONSTANT ] type [ NOT NULL ]
[ { DEFAULT | := } expression ];
```

La clause DEFAULT, si indiquée, spécifie la valeur initiale assignée à la variable quand on entre dans le bloc. Si la clause DEFAULT n'est pas indiquée, la variable est initialisée à la valeur SQL NULL. L'option CONSTANT empêche l'assignation de la variable, de sorte que sa valeur reste constante pour la durée du bloc. Si NOT NULL est spécifié, l'assignement d'une valeur NULL aboutira à une erreur d'exécution. Les valeurs par défaut de toutes les variables déclarées NOT NULL doivent être spécifiées non NULL. La valeur par défaut est évaluée à chaque entrée du bloc. Ainsi, par exemple, l'assignation de now() à une variable de type timestamp donnera à la variable l'heure de l'appel de la fonction courante, et non l'heure au moment où la fonction a été précompilée.

Exemples :

```
quantité integer DEFAULT 32;
url varchar := 'http://mysite.com';
id_utilisateur CONSTANT integer := 10;
```

1.2. Alias de paramètres de fonctions

Les paramètres passés aux fonctions sont nommés par les identifiants \$1, \$2, etc. Éventuellement, des alias peuvent être déclarés pour les noms de paramètres de type \$n afin d'améliorer la lisibilité. L'alias ou l'identifiant

numérique peuvent être utilisés indifféremment pour se référer à la valeur du paramètre. Il existe deux façons de créer un alias. La façon préférée est de donner un nom au paramètre dans la commande **CREATE FUNCTION**, par exemple :

```
CREATE FUNCTION taxe_ventes(sous_total
real) RETURNS real AS $$
BEGIN
    RETURN sous_total * 0.06;
END;
```

L'autre façon, la seule disponible pour les versions antérieures de PostgreSQL™ 8.0, est de déclarer explicitement un alias en utilisant la syntaxe de déclaration
nom ALIAS FOR \$n;
Le même exemple dans ce style ressemble à

```
CREATE FUNCTION taxe_ventes(real)
RETURNS real AS $$
DECLARE
    sous_total ALIAS FOR $1;
BEGIN
    RETURN sous_total * 0.06;
END;
$$ LANGUAGE plpgsql;
Quelques exemples de plus :
CREATE FUNCTION instr(varchar, integer)
RETURNS integer AS '
DECLARE
    v_string ALIAS FOR $1;
    index ALIAS FOR $2;
BEGIN
    -- quelques traitements utilisant ici v_string et
    index
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
concat_champs_selectionnees(in_t
un_nom_de_table) RETURNS text AS $$
BEGIN
    RETURN in_t.f1 || in_t.f3 || in_t.f5 || in_t.f7;
END;
$$ LANGUAGE plpgsql;
```

Quand une fonction PL/pgSQL est déclarée avec des paramètres en sortie, ces derniers se voient attribués les noms \$n et des alias optionnels de la même façon que les paramètres en entrée. Un paramètre en sortie est une variable qui commence avec la valeur NULL ; il devrait se voir attribuer une valeur lors de l'exécution de la

fonction. La valeur finale du paramètre est ce qui est renvoyée. Par exemple, l'exemple `taxe_ventes` peut s'écrire de cette façon :

```
CREATE FUNCTION taxe_ventes(sous_total
real, OUT taxe real) AS $$
BEGIN
    taxe := sous_total * 0.06;
END;
$$ LANGUAGE plpgsql;
```

Notez que nous avons omis `RETURNS real`. Nous aurions pu l'inclure mais cela aurait été redondant. Les paramètres en sortie sont encore plus utiles lors du retour de plusieurs valeurs. Un exemple trivial est :

```
CREATE FUNCTION somme_n_produits(x int, y
int, OUT somme int, OUT produit int) AS $$
BEGIN
    somme := x + y;
    produit := x * y;
END;
$$ LANGUAGE plpgsql;
```

Si une clause `RETURNS` est donnée, elle doit spécifier `RETURNS record`. Lorsque le type de retour d'une fonction PL/pgSQL est déclaré comme type polymorphe (anyelement ou anyarray), un paramètre spécial `$0` est créé. Son type de donnée est le type effectif de retour de la fonction, déduit d'après les types d'entrée. Ceci permet à la fonction d'accéder à son type de retour réel. `$0` est initialisé à `NULL` et peut être modifié par la fonction, de sorte qu'il peut être utilisé pour contenir la variable de retour si besoin est, bien que cela ne soit pas requis. On peut aussi donner un alias à `$0`. Par exemple, cette fonction s'exécute comme un opérateur `+` pour n'importe quel type de données.

```
CREATE FUNCTION ajoute_trois_valeurs(v1
anyelement, v2 anyelement, v3 anyelement)
RETURNS anyelement AS $$
DECLARE
    resultat ALIAS FOR $0;
BEGIN
    resultat := v1 + v2 + v3;
    RETURN resultat;
END;
$$ LANGUAGE plpgsql;
```

Le même effet peut être obtenu en déclarant un ou plusieurs paramètres en sortie comme `anyelement` ou `anyarray`. Dans ce cas, le paramètre spécial `$0` n'est pas utilisé ; les

paramètres en sortie servent ce même but. Par exemple :

```
CREATE FUNCTION ajoute_trois_valeurs(v1
anyelement, v2 anyelement, v3 anyelement,
OUT somme anyelement)
AS $$
BEGIN
    somme := v1 + v2 + v3;
END;
$$ LANGUAGE plpgsql;
```

1.3. Types ligne

```
nom nom_table%ROWTYPE;
nom nom_type_composite;
```

Une variable de type composite est appelée variable ligne (ou variable row-type). Une telle variable peut contenir une ligne entière de résultat de requête **SELECT** ou **FOR**, du moment que l'ensemble de colonnes de la requête correspond au type déclaré de la variable. Les champs individuels de la valeur row sont accessibles en utilisant la notation pointée, par exemple `varligne.champ`.

Une variable ligne peut être déclarée de façon à avoir le même type que les lignes d'une table ou d'une vue existante, en utilisant la notation `nom_table%ROWTYPE`. Elle peut aussi être déclarée en donnant un nom de type composite. Chaque table ayant un type de données associé du même nom, il importe peu dans PostgreSQL™ que vous écriviez `%ROWTYPE` ou pas. Cependant, la forme utilisant `%ROWTYPE` est plus portable.

Les paramètres d'une fonction peuvent être des types composites (lignes complètes de tables). Dans ce cas, l'identifiant correspondant `$n` sera une variable ligne à partir de laquelle les champs peuvent être sélectionnés avec la notation pointée, par exemple `$1.id_utilisateur`.

Seules les colonnes définies par l'utilisateur sont accessibles dans une variable de type ligne, et non l'OID ou d'autres colonnes systèmes (parce que la ligne pourrait être issue d'une vue). Les champs du type ligne héritent des tailles des champs de la table ou de leur précision pour les types de données tels que `char(n)`. Voici un exemple d'utilisation des types composites. `table1` et `table2` sont des tables ayant au moins les champs mentionnés :

```
CREATE FUNCTION assemble_champs(t_ligne
table1) RETURNS text AS $$
DECLARE
    t2_ligne table2%ROWTYPE;
BEGIN
    SELECT * INTO t2_ligne FROM table2
```

```

WHERE ... ;
    RETURN t_ligne.f1 || t2_ligne.f3 || t_ligne.f5 ||
t2_ligne.f7;
END;
$$ LANGUAGE plpgsql;

SELECT assemble_champs(t.*) FROM table1 t
WHERE ...
;

```

1.4. Types record

nom RECORD;

Les variables record sont similaires aux variables de type ligne mais n'ont pas de structure prédéfinie. Elles empruntent la structure effective de type ligne de la ligne à laquelle elles sont affectées durant une commande **SELECT** ou **FOR**. La sous-structure d'une variable record peut changer à chaque fois qu'on l'affecte. Une conséquence de cela est qu'elle n'a pas de sous-structure jusqu'à ce qu'elle ait été affectée, et toutes les tentatives pour accéder à un de ses champs entraînent une erreur d'exécution. Notez que RECORD n'est pas un vrai type de données mais seulement un paramètre fictif (placeholder). Il faut aussi réaliser que lorsqu'une fonction PL/pgSQL est déclarée renvoyer un type record, il ne s'agit pas tout à fait du même concept qu'une variable record, même si une telle fonction peut aussi utiliser une variable record pour contenir son résultat. Dans les deux cas, la structure réelle de la ligne n'est pas connue quand la fonction est écrite mais, dans le cas d'une fonction renvoyant un type record, la structure réelle est déterminée quand la requête appelante est analysée, alors qu'une variable record peut changer sa structure de ligne à la volée.

2. Expressions

Toutes les expressions utilisées dans les instructions PL/pgSQL sont traitées par l'exécuteur SQL classique du serveur. En effet, une requête comme

```
SELECT expression
```

est exécutée en utilisant le gestionnaire SPI. Avant l'évaluation, les occurrences des identifiants de variables PL/pgSQL sont remplacées par des paramètres, et les valeurs réelles des variables sont passées à l'exécuteur dans le tableau des paramètres. Ceci permet au plan de requêtes pour le **SELECT** de n'être élaboré qu'une fois et réutilisé pour les évaluations postérieures. L'évaluation faite par l'analyseur syntaxique principal de PostgreSQL™ a quelques effets de

bord sur l'interprétation des valeurs constantes. Plus précisément, il y a une différence entre ce que font ces deux fonctions :

```

CREATE FUNCTION trace1(texte_trace text)
RETURNS timestamp AS $$
BEGIN
    INSERT INTO table_trace VALUES
(texte_trace, 'now');
    RETURN 'now';
END;
$$ LANGUAGE plpgsql;
et
CREATE FUNCTION trace2(texte_trace text)
RETURNS timestamp AS $$
DECLARE
    heure timestamp;
BEGIN
    heure := 'now';
    INSERT INTO table_trace VALUES
(texte_trace, heure);
    RETURN heure;
END;
$$ LANGUAGE plpgsql;

```

Dans le cas de trace1, l'analyseur syntaxique principal de PostgreSQL™ sait, quand il élabore le plan pour l'**INSERT**, que la chaîne 'now' doit être interprétée comme un timestamp parce que la colonne cible de table_trace est de ce type. Ainsi, 'now' sera converti en une constante lors de l'instruction **INSERT**, puis utilisé dans tous les appels à trace1 pendant le temps que durera la session. Il va sans dire que ce n'est pas ce que le programmeur voulait.

Dans le cas de trace2, l'analyseur principal de PostgreSQL™ ne sait pas quel type 'now' doit devenir, et par conséquent, il renvoie une valeur de type text contenant la chaîne now. Durant l'assignation consécutive de la variable locale heure, l'interpréteur PL/pgSQL transtype cette chaîne en type timestamp en appelant les fonctions text_out et timestamp_in pour la conversion. Ainsi, l'horodateur est mis à jour à chaque exécution comme l'attend le programmeur.

La nature modifiable des variables record présente un problème lors de cette connexion. Quand les champs d'une variable record sont utilisés dans les expressions ou les instructions, les types de données des champs ne doivent pas changer entre les appels de deux expressions identiques, puisque l'expression sera planifiée en utilisant le type de données présent quand l'expression est atteinte pour la première fois. Gardez ceci à l'esprit quand vous écrivez des procédures déclencheurs qui gèrent des événements pour plus d'une table (**EXECUTE**

peut être utilisé pour contourner le problème si nécessaire).

4) Instructions de base

Dans cette section ainsi que les suivantes, nous décrirons tous les types d'instructions explicitement compris par PL/pgSQL. Tout ce qui n'est pas reconnu comme l'un de ces types d'instruction est présumé être une commande SQL et est envoyé au moteur principal de bases de données pour être exécutée (après substitution de chaque variable PL/pgSQL utilisée dans l'instruction). Ainsi, par exemple, les commandes SQL **INSERT**, **UPDATE** et **DELETE** peuvent être considérées comme des instructions de PL/pgSQL, mais ne sont pas spécifiquement listées ici.

4.1. Assignment

L'assignation d'une valeur à une variable ou à un champ row/record est écrite ainsi :
identifiant := expression;
Comme expliqué plus haut, l'expression dans une telle instruction est évaluée au moyen de la commande SQL **SELECT** envoyée au moteur principal de bases de données. L'expression ne doit manier qu'une seule valeur.
Si le type de données du résultat de l'expression ne correspond pas au type de donnée de la variable, ou que la variable a une taille ou une précision (comme char(20)), la valeur résultat sera implicitement convertie par l'interpréteur PL/pgSQL en utilisant la fonction d'écriture (output-fonction) du type du résultat, et la fonction d'entrée (input-fonction) du type de la variable. Notez que cela pourrait potentiellement conduire à des erreurs d'exécution générées par la fonction d'entrée si la forme de la chaîne de la valeur résultat n'est pas acceptable par la fonction d'entrée.

Exemples :
id_utilisateur := 20;
tax := sous_total * 0.06;

4.2. SELECT INTO

Le résultat d'une commande **SELECT** manipulant plusieurs colonnes (mais une seule ligne) peut être assignée à une variable de type record ou ligne ou une liste de valeurs scalaires. Ceci est fait via :
SELECT INTO cible expressions **FROM** ...;
où cible peut être une variable record, une variable ligne, ou une liste, séparées de virgules, de simples variables de champs record/ligne.

L'expression **select_expressions** et le reste de la commande sont identiques à du SQL standard. Notez que cela est assez différent de l'interprétation normale par PostgreSQL™ de **SELECT INTO**, où la cible de **INTO** est une table nouvellement créée. Si vous voulez créer une table à partir du résultat d'un **SELECT** d'une fonction PL/pgSQL, utilisez la syntaxe **CREATE TABLE ... AS SELECT**.

Si une ligne ou une liste de variable est utilisée comme cible, les valeurs sélectionnées doivent correspondre exactement à la structure de la cible, sinon une erreur d'exécution se produira. Quand une variable record est la cible, elle se configure seule automatiquement au type ligne formé par les colonnes résultant de la requête. À l'exception de la clause **INTO**, l'instruction **SELECT** est identique à la commande SQL **SELECT** normale et peut en utiliser toute la puissance.

La clause **INTO** peut apparaître pratiquement partout dans l'instruction **SELECT**. De façon personnalisé, il est écrit soit juste après **SELECT** comme indiqué ci-dessus soit juste avant **FROM** -- c'est-à-dire soit juste avant soit juste après la liste de **select_expressions**.
Si la requête ne renvoie aucune ligne, des valeurs **NULL** sont assignées au(x) cible(s). Si la requête renvoie plusieurs lignes, la première ligne est assignée au(x) cible(s) et le reste est rejeté (notez que « la première ligne » n'est pas correctement définie à moins d'utiliser **ORDER BY**).

Actuellement, la clause **INTO** peut apparaître presque n'importe où dans l'instruction **SELECT** mais il est recommandé de la placer immédiatement après le mot clé **SELECT** comme décrit plus haut. Les versions futures de PL/pgSQL pourront être moins laxistes sur le placement de la clause **INTO**.

Vous pouvez vérifier la variable spéciale **FOUND** après une instruction **SELECT INTO** pour déterminer si l'affectation est réussie, c'est-à-dire si au moins une ligne a été renvoyée par la requête. Par exemple :

```
SELECT INTO mon_enreg * FROM emp WHERE
nomemp = mon_nom;
IF NOT FOUND THEN
    RAISE EXCEPTION 'employé % non trouvé',
mon_nom;
END IF;
```

Pour tester si un résultat record/ligne est **NULL**, vous pouvez utiliser la conditionnelle **IS NULL**. Il n'y a cependant aucun moyen de dire si une ou plusieurs lignes additionnelles ont été rejetées.

Voici un exemple qui traite le cas où aucune ligne n'a été renvoyée.

```
DECLARE
  enreg_utilisateurs RECORD;
BEGIN
  SELECT INTO enreg_utilisateurs * FROM
  utilisateurs WHERE id_utilisateur=3;

  IF enreg_utilisateurs.accueil IS NULL THEN
    -- l'utilisateur n'a entré aucune page,
    renvoyer "http://"
    RETURN 'http://';
  END IF;
END;
```

4.3. Exécuter une expression ou requête sans résultat

Quelque fois, on souhaite évaluer une expression ou une requête mais rejeter le résultat (généralement parce qu'on appelle une fonction qui a des effets de bords utiles mais pas de résultat utile). Pour ce faire dans PL/pgSQL, utilisez l'instruction **PERFORM** :

PERFORM requête;

Ceci exécute requête et annule le résultat. Écrivez requête de la même façon que vous le feriez dans une commande SQL **SELECT** mais remplacez le mot clé initial **SELECT** par **PERFORM**. Les variables PL/pgSQL seront substituées dans la requête comme d'habitude. Par ailleurs, la variable spéciale FOUND est positionnée à true si la requête produit au moins une ligne ou false si elle n'en produit aucune.

Note

On pourrait s'attendre à ce qu'un **SELECT** sans clause INTO aboutisse à ce résultat, mais en réalité la seule façon acceptée de faire cela est **PERFORM**.

Un exemple :

```
PERFORM
creer_vuemat('cs_session_page_requests_mv',
ma_requete);
```

4.4. Ne rien faire du tout

Quelque fois, une instruction qui ne fait rien est utile. Par exemple, il indique qu'une partie de la chaîne if/then/else est délibérément vide. Pour cela, utilisez l'instruction :

NULL;

Par exemple, les deux fragments de code suivants sont équivalents :

```
BEGIN
  y := x / 0;
EXCEPTION
  WHEN division_by_zero THEN
    NULL; -- ignore l'erreur
END;

BEGIN
  y := x / 0;
EXCEPTION
  WHEN division_by_zero THEN -- ignore l'erreur
  END;
```

Ce qui est préférable est une histoire de goût.

Note

Dans le PL/SQL d'Oracle, les listes d'instructions vides ne sont pas autorisées et, du coup, les instructions **NULL** sont *requis* dans les situations telles que celles-ci. PL/pgSQL vous permet d'écrire simplement rien.

4.5. Obtention du statut du résultat

Il y a plusieurs moyens pour déterminer l'effet d'une commande. La première méthode est d'utiliser **GET DIAGNOSTICS** :

```
GET DIAGNOSTICS variable = élément [ , ... ] ;
```

Cette commande permet la récupération des indicateurs d'état du système. Chaque élément est un mot clé identifiant une valeur d'état devant être affectée à la variable indiquée (qui doit être du bon type de donnée pour que l'affectation puisse se faire sans erreur.) Les éléments d'état actuellement disponibles sont ROW_COUNT, le nombre de lignes traitées par la dernière commande SQL envoyée au moteur SQL, et RESULT_OID, l'OID de la dernière ligne insérée par la commande SQL la plus récente. Notez que RESULT_OID n'est utile qu'après une commande **INSERT** dans une table contenant des OID.

Exemple :

```
GET DIAGNOSTICS var_entier = ROW_COUNT;
```

La seconde méthode permettant de déterminer les effets d'une commande est la variable spéciale nommée FOUND de type boolean. La variable FOUND est initialisée à false au début de chaque fonction PL/pgSQL. Elle est positionnée par chacun des types d'instructions suivants :

- Une instruction **SELECT INTO** positionne FOUND à true si elle renvoie une ligne, false si aucune ligne n'est renvoyée.

- Une instruction **PERFORM** positionne FOUND à true si elle produit (rejette) une ligne, faux si aucune ligne n'est produite.
- Les instructions **UPDATE**, **INSERT**, et **DELETE** positionnent FOUND à true si au moins une ligne est affectée, false si aucune ligne n'est affectée.
- Une instruction **FETCH** positionne FOUND à true si elle renvoie une ligne, false si aucune ligne n'est renvoyée.
- La commande **FOR** positionne FOUND à true si elle effectue une itération une ou plusieurs fois, sinon elle renvoie false. Ceci s'applique aux trois variantes de l'instruction **FOR** (boucles **FOR** integer, **FOR** record-set, et **FOR** record-set dynamique). FOUND n'est positionnée de cette façon que quand la boucle **FOR** s'achève ; dans l'exécution de la chaîne, FOUND n'est pas modifiée par l'instruction **FOR**, bien qu'elle puisse être modifiée par l'exécution d'autres instructions situées dans le corps de la boucle.

FOUND est une variable locale à l'intérieur de chaque fonction PL/pgSQL ; chaque changement qui y est fait n'affecte que la fonction en cours.

5. Structures de contrôle

Les structures de contrôle sont probablement la partie la plus utile (et importante) de PL/pgSQL. Grâce aux structures de contrôle de PL/pgSQL, vous pouvez manipuler les données PostgreSQL™ de façon très flexible et puissante.

5.1. Retour d'une fonction

Il y a deux commandes disponibles qui vous permettent de renvoyer des données d'une fonction : **RETURN** et **RETURN NEXT**.

5.1.1. RETURN

RETURN expression;

RETURN accompagné d'une expression termine la fonction et renvoie le valeur de l'expression à l'appelant. Cette forme est à utiliser avec des fonctions PL/pgSQL qui ne renvoient pas d'ensemble de valeurs.

Lorsqu'elle renvoie un type scalaire, n'importe quelle expression peut être utilisée. Le résultat de l'expression sera automatiquement converti vers le type de retour de la fonction, comme décrit pour les affectations. Pour renvoyer une valeur composite (ligne), vous devez écrire une variable record ou ligne comme expression.

Si vous déclarez la fonction avec des paramètres en sortie, écrivez seulement **RETURN** sans

expression. Les valeurs courantes des paramètres en sortie seront renvoyées.

Si vous déclarez que la fonction renvoie void, une instruction **RETURN** peut être utilisée pour quitter rapidement la fonction ; mais n'écrivez pas d'expression après **RETURN**.

La valeur de retour d'une fonction ne peut pas être laissée indéfinie. Si le contrôle atteint la fin du bloc de haut niveau de la fonction, sans parvenir à une instruction **RETURN**, une erreur d'exécution survient. Néanmoins, cette restriction ne s'applique pas aux fonctions sans paramètre de sortie et aux fonctions renvoyant void. Dans ces cas, une instruction **RETURN** est automatiquement exécutée si le bloc de haut niveau est terminé.

5.1.2. RETURN NEXT

RETURN NEXT expression;

Lorsqu'une fonction PL/pgSQL est déclarée renvoyer SETOF type quelconque, la procédure à suivre est légèrement différente. Dans ce cas, les éléments individuels à renvoyer sont spécifiés dans les commandes **RETURN NEXT**, et ensuite une commande **RETURN** finale sans arguments est utilisée pour indiquer que la fonction a terminé son exécution. **RETURN NEXT** peut être utilisé avec des types scalaires et des types composites de données ; avec un type de résultat composite, une « table » entière de résultats sera renvoyée.

RETURN NEXT ne sort pas vraiment de la fonction -- il met simplement de côté la valeur de l'expression. Puis, l'exécution continue avec la prochaine instruction de la fonction PL/pgSQL. Au fur et à mesure que des commandes **RETURN NEXT** successives sont exécutées, l'ensemble de résultat est construit. Un **RETURN** final, qui pourrait ne pas avoir d'argument, fait sortir de la fonction (mais vous pouvez aussi atteindre la fin de la fonction).

Si vous avez déclaré la fonction avec des paramètres en sortie, écrivez simplement **RETURN NEXT** sans expression. Les valeurs en cours de(s) paramètre(s) en sortie seront sauvegardées pour un retour éventuel. Notez que vous devez déclarer la fonction comme renvoyant SETOF record s'il y a plusieurs paramètres en sortie ou SETOF untype quand il y a un seul paramètre en sortie de type untype, pour créer une fonction renvoyant un ensemble avec les paramètres en sortie.

Les fonctions qui utilisent **RETURN NEXT** devraient être appelées d'après le modèle suivant :

```
SELECT * FROM une_fonction();
```

En fait, la fonction doit être utilisée comme une table source dans une clause FROM.

Note

L'implémentation actuelle de **RETURN NEXT** pour PL/pgSQL emmagasine la totalité de l'ensemble des résultats avant d'effectuer le retour de la fonction, comme vu plus haut. Cela signifie que si une fonction PL/pgSQL produit une structure résultat très grande, les performances peuvent être faibles : les données seront écrites sur le disque pour éviter un épuisement de la mémoire mais la fonction en elle-même ne renverra rien jusqu'à ce que l'ensemble des résultats entier soit généré. Une version future de PL/pgSQL pourra permettre aux utilisateurs de définir des fonctions renvoyant des ensembles qui n'auront pas cette limitation. Actuellement, le point auquel les données commencent à être écrites sur le disque est contrôlé par la variable de configuration work_mem. Les administrateurs ayant une mémoire suffisante pour enregistrer des ensembles de résultats plus importants en mémoire devraient envisager l'augmentation de ce paramètre.

a. Contrôles conditionnels

Les instructions IF vous permettent d'exécuter des commandes basées sur certaines conditions. PL/pgSQL a cinq formes de IF :

- IF ... THEN
- IF ... THEN ... ELSE
- IF ... THEN ... ELSE IF
- IF ... THEN ... ELSIF ... THEN ... ELSE
- IF ... THEN ... ELSEIF ... THEN ... ELSE

5.2.1. IF-THEN

```
IF expression-booleenne THEN
    instructions
END IF;
```

Les instructions IF-THEN sont la forme la plus simple de IF. Les instructions entre THEN et END IF seront exécutées si la condition est vraie. Autrement, elles seront ignorées.

Exemple :

```
IF v_id_utilisateur <> 0 THEN
    UPDATE utilisateurs SET email = v_email
    WHERE id_utilisateur = v_id_utilisateur;
END IF;
```

5.2.2. IF-THEN-ELSE

```
IF expression-booleenne THEN
    instructions
ELSE
    instructions
END IF;
```

Les instructions IF-THEN-ELSE s'ajoutent au IF-THEN en vous permettant de spécifier un autre ensemble d'instructions à exécuter si la condition est fausse.

Exemples :

```
IF id_parent IS NULL OR id_parent = ''
THEN
    RETURN nom_complet;
ELSE
    RETURN hp_true_filename(id_parent) || ' ' ||
    nom_complet;
END IF;
IF v_nombre > 0 THEN
    INSERT INTO nombre_utilisateurs (nombre)
VALUES (v_nombre);
    RETURN 't';
ELSE
    RETURN 'f';
END IF;
```

5.2.3. IF-THEN-ELSE IF

Les instructions IF peuvent être imbriquées, comme dans l'exemple suivant :

```
IF demo_ligne.sexe = 'm' THEN
    texte_sexe := "homme";
ELSE
    IF demo_ligne.sexe = 'f' THEN
        texte_sexe := 'femme';
    END IF;
END IF;
```

Lorsque vous utilisez cette forme, vous imbriquez une instruction IF dans la partie ELSE d'une instruction IF extérieure. Ainsi, vous avez besoin d'une instruction END IF pour chaque IF imbriqué et une pour le IF-ELSE parent. Ceci fonctionne mais devient fastidieux quand il y a de nombreuses alternatives à traiter. Considérez alors la forme suivante.

5.2.4. IF-THEN-ELSIF-ELSE

```
IF expression-booleenne THEN
    instructions
[ ELSEIF expression-booleenne THEN
    instructions
[ ELSEIF expression-booleenne THEN
    instructions
    ...
]
]
[ ELSE
    instructions ]
END IF;
```

IF-THEN-ELSIF-ELSE fournit une méthode plus pratique pour vérifier de nombreuses alternatives

en une instruction. Elle est équivalente formellement aux commandes IF-THEN-ELSE-IF-THEN imbriquées, mais un seul END IF est nécessaire.

Voici un exemple :

```
IF nombre = 0 THEN
    resultat := 'zero';
ELSIF nombre > 0 THEN
    resultat := 'positif';
ELSIF nombre < 0 THEN
    resultat := 'negatif';
ELSE
    -- hmm, la seule possibilité est que le nombre
    soit NULL
    resultat := 'NULL';
END IF;
```

5.2.5. IF-THEN-ELSEIF-ELSE

ELSEIF est un alias pour ELSIF.

5.2.6. Boucles simples

Grâce aux instructions LOOP, EXIT, CONTINUE, WHILE et FOR, vous pouvez faire en sorte que vos fonctions PL/pgSQL répètent une série de commandes.

5.2.6.1. LOOP

```
[<<label>>]
LOOP
    instructions
END LOOP [ label ];
```

LOOP définit une boucle inconditionnelle répétée indéfiniment jusqu'à ce qu'elle soit terminée par une instruction EXIT ou **RETURN**. Le label optionnel peut être utilisé par les instructions EXIT et CONTINUE dans le cas de boucles imbriquées pour définir la boucle impliquée.

5.2.6.2. EXIT

EXIT [label] [WHEN expression];

Si aucun label n'est donné, la boucle la plus imbriquée se termine et l'instruction suivant END LOOP est exécutée. Si un label est donné, ce doit être le label de la boucle, du bloc courant ou d'un niveau moins imbriqué. La boucle ou le bloc nommé se termine alors et le contrôle continue avec l'instruction située après le END de la boucle ou du bloc correspondant.

Si WHEN est spécifié, la sortie de boucle ne s'effectue que si expression vaut true. Sinon, le contrôle passe à l'instruction suivant le EXIT. EXIT peut être utilisé pour tous les types de boucles ; il n'est pas limité aux boucles non conditionnelles. Lorsqu'il est utilisé avec un bloc

BEGIN, EXIT passe le contrôle à la prochaine instruction après la fin du bloc.

Exemples :

```
LOOP
    -- quelques traitements
    IF nombre > 0 THEN
        EXIT; -- sortie de boucle
    END IF;
END LOOP
LOOP
    -- quelques traitements
    EXIT WHEN nombre > 0;
END LOOP;

BEGIN
    -- quelques traitements
    IF stocks > 100000 THEN
        EXIT; -- cause la sortie (EXIT) du bloc
    BEGIN
        END IF;
    END;
```

;

5.2.6.3. CONTINUE

CONTINUE [label] [WHEN expression];

Si aucun label n'est donné, la prochaine itération de la boucle interne est commencée. C'est-à-dire que le contrôle est redonné à l'expression de contrôle de la boucle (s'il y en a une) et le corps de la boucle est ré-évaluée. Si le label est présent, il spécifie le label de la boucle dont l'exécution va être continué.

Si WHEN est spécifié, la prochaine itération de la boucle est commencée seulement si l'expression vaut true. Sinon, le contrôle est passé à l'instruction après CONTINUE.

CONTINUE peut être utilisé avec tous les types de boucles ; il n'est pas limité à l'utilisation des boucles inconditionnelles.

Exemples :

```
LOOP
    -- quelques traitements
    EXIT WHEN nombre > 100;
    CONTINUE WHEN nombre < 50;
    -- quelques traitements pour nombre IN [50 ..
    100]
END LOOP;
```

5.2.6.4. WHILE

```
[<<label>>]
WHILE expression LOOP
    instructions
END LOOP [ label ];
```

L'instruction WHILE répète une séquence d'instructions aussi longtemps que l'expression conditionnelle est évaluée à vrai. La condition est vérifiée juste avant chaque entrée dans le corps de la boucle.

Par exemple :

```
WHILE montant_possede > 0 AND
balance_cadeau > 0 LOOP
    -- quelques traitements ici
END LOOP;
```

```
WHILE NOT expression_booleenne LOOP
    -- quelques traitements ici
END LOOP;
```

5.2.6.5. FOR (variante avec entier)

```
[<<label>>]
FOR nom IN [ REVERSE ] expression ..
expression LOOP
    instruction
END LOOP [ label ];
```

Cette forme de FOR crée une boucle qui effectue une itération sur une plage de valeurs entières. La variable nom est automatiquement définie comme un type integer et n'existe que dans la boucle. Les deux expressions donnant les limites inférieures et supérieures de la plage sont évaluées une fois en entrant dans la boucle. Le pas de l'itération est normalement de 1 mais vaut -1 quand REVERSE est spécifié.

Quelques exemples de boucles FOR avec entiers :

```
FOR i IN 1..10 LOOP
    -- quelques calculs ici
    RAISE NOTICE 'i is %', i;
END LOOP;
```

```
FOR i IN REVERSE 10..1 LOOP
    -- quelques calculs ici
END LOOP;
```

Si la limite basse est plus grande que la limite haute (ou moins grande dans le cas du REVERSE), le corps de la boucle n'est pas exécuté du tout. Aucune erreur n'est renvoyée.

5.2.6.6. Boucler dans les résultats de requêtes

En utilisant un type de FOR différent, vous pouvez itérer au travers des résultats d'une requête et par là-même manipuler ces données. La syntaxe est la suivante :

```
[<<label>>]
FOR record_ou_ligne IN requête LOOP
    instructions
END LOOP [ label ];
```

La variable record ou ligne est successivement assignée à chaque ligne résultant de la requête (qui doit être une commande **SELECT**) et le corps de la boucle est exécuté pour chaque ligne. Voici un exemple :

```
CREATE FUNCTION cs_refresh_mvviews()
RETURNS integer AS $$
DECLARE
    mvviews RECORD;
BEGIN
    PERFORM cs_log('Refreshing materialized
views...');

    FOR mvviews IN SELECT * FROM
cs_materialized_views ORDER BY sort_key
LOOP

        -- À présent "mvviews" contient un
enregistrement de cs_materialized_views

        PERFORM cs_log('Refreshing materialized
view ' || quote_ident(mvviews.mv_name) || '...');
        EXECUTE 'TRUNCATE TABLE ' ||
quote_ident(mvviews.mv_name);
        EXECUTE 'INSERT INTO ' ||
quote_ident(mvviews.mv_name) || ' ' ||
mvviews.mv_query;
    END LOOP;

    PERFORM cs_log('Done refreshing
materialized views.');
```

Si la boucle est terminée par une instruction EXIT, la dernière valeur ligne assignée est toujours accessible après la boucle. L'instruction FOR-IN-EXECUTE est un moyen d'itérer sur des lignes :

```
[<<label>>]
FOR record_ou_ligne IN EXECUTE
expression_texte LOOP
    instructions
END LOOP [ label ];
```

Ceci est identique à la forme précédente, à ceci près que l'expression **SELECT** source est spécifiée comme une expression chaîne, évaluée et replanifiée à chaque entrée dans la boucle FOR. Ceci permet au développeur de choisir entre la vitesse d'une requête préplanifiée et la flexibilité d'une requête dynamique, uniquement avec l'instruction **EXECUTE**.

Note

L'analyseur PL/pgSQL distingue actuellement deux types de boucles FOR (entier ou résultat d'une requête) en vérifiant si .. apparaît à

l'extérieur des parenthèses entre IN et LOOP. Si .. n'est pas trouvé, la boucle est supposée être une boucle entre des lignes. Une mauvaise saisie de .. amènera donc une plainte du type « loop variable of loop over rows must be a record or row variable » (NdT : une variable de boucle d'une boucle sur des enregistrements doit être un enregistrement ou une variable de type ligne) plutôt qu'une simple erreur de syntaxe comme vous pourriez vous y attendre.

6. Récupérer les erreurs

Par défaut, toute erreur survenant dans une fonction PL/pgSQL annule l'exécution de la fonction mais aussi de la transaction qui l'entoure. Vous pouvez récupérer les erreurs en utilisant un bloc **BEGIN** avec une clause EXCEPTION. La syntaxe est une extension de la syntaxe habituelle pour un bloc **BEGIN** :

```
[ <<label>> ]
[ DECLARE
  declarations ]
BEGIN
instructions
EXCEPTION
WHEN condition [ OR condition ... ] THEN
instructions_gestion_erreurs
[ WHEN condition [ OR condition ... ] THEN
  instructions_gestion_erreurs
... ]
END;
```

Si aucune erreur ne survient, cette forme de bloc exécute simplement toutes les instructions puis passe le contrôle à l'instruction suivant END. Mais si une erreur survient à l'intérieur des instructions, le traitement en cours des instructions est abandonné et le contrôle est passé à la liste d'EXCEPTION. Une recherche est effectuée sur la liste pour la première condition correspondant à l'erreur survenue. Si une correspondance est trouvée, les instructions_gestion_erreurs correspondantes sont exécutées puis le contrôle est passé à l'instruction suivant le END. Si aucune correspondance n'est trouvée, l'erreur se propage comme si la clause EXCEPTION n'existait pas du tout : l'erreur peut être récupérée par un bloc l'enfermant avec EXCEPTION ou, s'il n'existe pas, elle annule le traitement de la fonction. Si une nouvelle erreur survient à l'intérieur des instructions_gestion_erreurs sélectionnées, elle ne peut pas être récupérée par cette clause EXCEPTION mais est propagée en dehors. Une clause EXCEPTION l'englobant pourrait la récupérer.

Quand une erreur est récupérée par une clause EXCEPTION, les variables locales de la fonction PL/pgSQL restent dans le même état qu'au moment où l'erreur est survenue mais toutes les

modifications à l'état persistant de la base de données à l'intérieur du bloc sont annulées. Comme exemple, considérez ce fragment :

```
INSERT INTO mon_tableau(prenom, nom)
VALUES('Tom', 'Jones');
BEGIN
  UPDATE mon_tableau SET prenom = 'Joe'
  WHERE nom = 'Jones';
  x := x + 1;
  y := x / 0;
  EXCEPTION
    WHEN division_by_zero THEN
      RAISE NOTICE 'récupération de l'erreur
division_by_zero';
  RETURN x;
END;
```

Quand le contrôle parvient à l'affectation de y, il échouera avec une erreur division_by_zero. Elle sera récupérée par la clause EXCEPTION. La valeur renvoyée par l'instruction **RETURN** sera la valeur incrémentée de x mais les effets de la commande **UPDATE** auront été annulés. La commande **INSERT** précédant le bloc ne sera pas annulée, du coup le résultat final est que la base de données contient Tom Jones et non pas Joe Jones.

Exemple 1. Exceptions avec UPDATE/INSERT

Cet exemple utilise un gestionnaire d'exceptions pour réaliser soit un **UPDATE** soit un **INSERT**, comme approprié.

```
CREATE TABLE base (a INT PRIMARY KEY, b
TEXT);
```

```
CREATE FUNCTION fusionne_base(cle INT, donnee
TEXT) RETURNS VOID AS
$$
BEGIN
  LOOP
    UPDATE base SET b = donnee WHERE a = cle;
    IF found THEN
      RETURN;
    END IF;

    BEGIN
      INSERT INTO base(a,b) VALUES (cle,
donnee);
      RETURN;
    EXCEPTION WHEN unique_violation THEN
      -- do nothing
    END;
  END LOOP;
END;
$$
LANGUAGE plpgsql;

SELECT fusionne_base(1, 'david');
SELECT fusionne_base(1, 'dennis');
```

7. Curseurs

Plutôt que d'exécuter la totalité d'une requête à la fois, il est possible de créer un curseur qui encapsule la requête, puis en lit le résultat quelques lignes à la fois. Une des raisons pour faire de la sorte est d'éviter les surcharges de mémoire quand le résultat contient un grand nombre de lignes (cependant, les utilisateurs PL/pgSQL n'ont généralement pas besoin de se préoccuper de cela puisque les boucles FOR utilisent automatiquement un curseur en interne pour éviter les problèmes de mémoire). Un usage plus intéressant est de renvoyer une référence à un curseur qu'une fonction a créé, permettant à l'appelant de lire les lignes. C'est un moyen efficace de renvoyer de grands ensembles de lignes à partir des fonctions.

7.1. Déclaration de variables curseur

Tous les accès aux curseurs dans PL/pgSQL se font par les variables curseur, qui sont toujours du type de données spécial `refcursor`. Un des moyens de créer une variable curseur est de simplement la déclarer comme une variable de type `refcursor`. Un autre moyen est d'utiliser la syntaxe de déclaration de curseur qui est en général :

```
nom CURSOR [ ( arguments ) ] FOR requête ;
```

(FOR peut être remplacé par IS pour la compatibilité avec Oracle™). arguments, si spécifié, est une liste de paires de nom type-de-donnée qui définit les noms devant être remplacés par les valeurs des paramètres dans la requête donnée. La valeur effective à substituer pour ces noms sera spécifiée plus tard lors de l'ouverture du curseur.

Quelques exemples :

```
DECLARE
  curs1 refcursor;
  curs2 CURSOR FOR SELECT * FROM tenk1;
  curs3 CURSOR (cle integer) IS SELECT *
FROM tenk1 WHERE unique1 = cle;
```

Ces variables sont toutes trois du type de données `refcursor` mais la première peut être utilisée avec n'importe quelle requête alors que la seconde a une requête complètement spécifiée qui lui est déjà liée, et la dernière est liée à une requête paramétrée (cle sera remplacée par un paramètre de valeur entière lors de l'ouverture du curseur). La variable `curs1` est dite non liée puisqu'elle n'est pas liée à une requête particulière.

7.2. Ouverture de curseurs

Avant qu'un curseur puisse être utilisé pour rapatrier des lignes, il doit être ouvert (c'est l'action équivalente de la commande SQL **DECLARE CURSOR**). PL/pgSQL dispose de trois formes pour l'instruction **OPEN**, dont deux utilisent des variables curseur non liées et la dernière une variable curseur liée.

7.2.1. OPEN FOR requête

OPEN curseur_non_lie FOR requête;
La variable curseur est ouverte et reçoit la requête spécifiée à exécuter. Le curseur ne peut pas être déjà ouvert, et il doit avoir été déclaré comme curseur non lié (c'est-à-dire comme une simple variable `refcursor`). La requête doit être un **SELECT** ou quelque chose d'autre qui renvoie des lignes (comme **EXPLAIN**). La requête est traitée de la même façon que les autres commandes SQL dans PL/pgSQL : les noms de variables PL/pgSQL sont substitués et le plan de requête est mis en cache pour une possible ré-utilisation.

Exemple :

```
OPEN curs1 FOR SELECT * FROM foo WHERE
cle = ma_cle
;
```

7.2.2. OPEN FOR EXECUTE

OPEN curseur_non_lié FOR EXECUTE chaîne_requête;

La variable curseur est ouverte et reçoit la requête spécifiée à exécuter. Le curseur ne peut pas être déjà ouvert et il doit avoir été déclaré comme curseur non-lié (c'est-à-dire comme une simple variable `refcursor`). La requête est spécifiée comme une expression chaîne de la même façon que dans une commande **EXECUTE**. Comme d'habitude, ceci donne assez de flexibilité pour que la requête puisse changer d'une exécution à l'autre.

Exemple :

```
OPEN curs1 FOR EXECUTE 'SELECT * FROM '
|| quote_ident($1);
```

7.2.3. Ouverture d'un curseur lié

OPEN curseur_lié [(arguments)];

Cette forme d'**OPEN** est utilisée pour ouvrir une variable curseur à laquelle la requête est liée au moment de la déclaration. Le curseur ne peut pas être déjà ouvert. Une liste des expressions arguments doit apparaître si et seulement si le curseur a été déclaré comme acceptant des arguments. Ces valeurs seront remplacées dans la requête. Le plan de requête pour un curseur lié est toujours considéré comme pouvant être mis

en cache ; il n'y a pas d'équivalent de la commande **EXECUTE** dans ce cas.

Exemples :

OPEN curs2;

OPEN curs3(42);

7.3. Utilisation des curseurs

Une fois qu'un curseur a été ouvert, il peut être manipulé grâce aux instructions décrites ci-dessous.

Ces manipulations n'ont pas besoin de se dérouler dans la même fonction que celle qui a ouvert le curseur. Vous pouvez renvoyer une valeur refcursor à partir d'une fonction et laisser l'appelant opérer sur le curseur (d'un point de vue interne, une valeur refcursor est simplement la chaîne de caractères du nom d'un portail contenant la requête active pour le curseur. Ce nom peut être passé à d'autres, affecté à d'autres variables refcursor et ainsi de suite, sans déranger le portail).

Tous les portails sont implicitement fermés à la fin de la transaction. C'est pourquoi une valeur refcursor est utilisable pour référencer un curseur ouvert seulement jusqu'à la fin de la transaction.

7.3.1. FETCH

FETCH curseur INTO cible;

FETCH rapatrie le rang suivant depuis le curseur dans une cible, qui peut être une variable ligne, une variable record ou une liste de simples variables séparées d'une virgule, exactement comme **SELECT INTO**. Comme pour **SELECT INTO**, la variable spéciale FOUND peut être vérifiée pour voir si une ligne a été obtenue ou pas.

Exemple :

FETCH curs1 INTO var_ligne;

FETCH curs2 INTO foo, bar, baz;

7.3.2. CLOSE

CLOSE curseur;

CLOSE ferme le portail sous-tendant un curseur ouvert. Ceci peut être utilisé pour libérer des ressources avant la fin de la transaction ou pour libérer la variable curseur pour pouvoir la réouvrir.

Exemple :

CLOSE curs1;

7.3.3. Renvoi de curseurs

Les fonctions PL/pgSQL peuvent renvoyer des curseurs à l'appelant. Ceci est utile pour renvoyer plusieurs lignes ou colonnes, spécialement avec des ensembles de résultats très grands. Pour

cela, la fonction ouvre le curseur et renvoie le nom du curseur à l'appelant (ou simplement ouvre le curseur en utilisant un nom de portail spécifié par ou autrement connu par l'appelant). L'appelant peut alors récupérer les lignes à partir du curseur. Le curseur peut être fermé par l'appelant ou il sera fermé automatiquement à la fin de la transaction.

Le nom du portail utilisé pour un curseur peut être spécifié par le développeur ou peut être généré automatiquement. Pour spécifier un nom de portail, affectez simplement une chaîne à la variable refcursor avant de l'ouvrir. La valeur de la variable refcursor sera utilisée par **OPEN** comme nom du portail sous-jacent. Néanmoins, si la variable refcursor est NULL, **OPEN** génère automatiquement un nom qui n'entre pas en conflit avec tout portail existant et l'affecte à la variable refcursor.

Note

Une variable curseur avec limites est initialisée avec la valeur de la chaîne représentant son nom, de façon à ce que le nom du portail soit identique au nom de la variable curseur, sauf si le développeur le surcharge par affectation avant d'ouvrir le curseur. Mais, une variable curseur sans limite aura par défaut la valeur NULL, dont il reçoit un nom unique généré automatiquement sauf s'il est surchargé.

L'exemple suivant montre une façon de fournir un nom de curseur par l'appelant :

```
CREATE TABLE test (col text);
INSERT INTO test VALUES ('123');
```

```
CREATE FUNCTION
fonction_reference(refcursor) RETURNS
refcursor AS '
BEGIN
    OPEN $1 FOR SELECT col FROM test;
    RETURN $1;
END;
' LANGUAGE plpgsql;
```

```
BEGIN;
SELECT fonction_reference('curseur_fonction');
FETCH ALL IN curseur_fonction;
COMMIT;
L'exemple suivant utilise la génération
automatique du nom du curseur :
CREATE FUNCTION fonction_reference2()
RETURNS refcursor AS '
DECLARE
    ref refcursor;
BEGIN
    OPEN ref FOR SELECT col FROM test;
    RETURN ref;
END;
' LANGUAGE plpgsql;
```

```
BEGIN;
SELECT fonction_reference2();
```

```
fonction_reference2
-----
<unnamed cursor 1>
(1 row)
```

```
FETCH ALL IN "<unnamed cursor 1>";
COMMIT;
```

L'exemple suivant montre une façon de renvoyer plusieurs curseurs à une seule fonction :

```
CREATE FUNCTION ma_fonction(refcursor,
refcursor) RETURNS SETOF refcursor AS $$
BEGIN
    OPEN $1 FOR SELECT * FROM table_1;
    RETURN NEXT $1;
    OPEN $2 FOR SELECT * FROM table_2;
    RETURN NEXT $2;
END;
$$ LANGUAGE plpgsql;

-- doit être dans une transaction pour utiliser les
-- curseurs.
BEGIN;

SELECT * FROM ma_fonction('a', 'b');

FETCH ALL FROM a;
FETCH ALL FROM b;
COMMIT;
```

8. LS TRIGGERS

Définition d'un trigger

Un trigger est un déclencheur associé à la table spécifiée et exécute une fonction *nomfonc* lorsque certains événements surviennent.

- Pour définir un nouveau déclencheur,

```
CREATE TRIGGER nom { BEFORE | AFTER } {
evenement [ OR ... ] }
ON table [ FOR [ EACH ] { ROW |
STATEMENT } ]
EXECUTE PROCEDURE nomfonc (
arguments )
```

Description

- Paramètres

nom : nom du nouveau déclencheur

BEFORE, AFTER : Détermine si la fonction est appelée avant ou après l'événement.

événement : Peut-être **INSERT**, **UPDATE** ou **DELETE** ; précise l'événement qui active le déclencheur. Plusieurs événements peuvent être précisés en les séparant par OR.

table : le nom (éventuellement qualifié du nom du schéma) de la table à laquelle est rattaché le déclencheur.

FOR EACH ROW, FOR EACH STATEMENT : Précise si la procédure du déclencheur doit être lancée pour chaque ligne affectée par l'événement ou simplement pour chaque instruction SQL. FOR EACH STATEMENT est la valeur par défaut.

nomfonc : une fonction utilisateur, déclarée sans argument et renvoyant le type trigger, exécutée à l'activation du déclencheur.

arguments : une liste optionnelle d'arguments séparés par des virgules à fournir à la fonction lors de l'activation du déclencheur.

Notes

Pour créer un déclencheur sur une table, l'utilisateur doit posséder le droit TRIGGER sur la table.

- Pour supprimer la définition d'un déclencheur

DROP TRIGGER [IF EXISTS] *nom* ON *table*

Seul le propriétaire de la table sur laquelle le déclencheur est défini peut exécuter cette commande.

Exemples

Destruction du déclencheur si_dist_existe de la table Client :

```
DROP TRIGGER si_dist ON Client;
```

Procédures trigger

PL/pgSQL peut être utilisé pour définir des procédures trigger. Une procédure trigger est créée grâce à la commande **CREATE FUNCTION** utilisée comme fonction sans arguments ayant un type de retour trigger. Notez que la fonction doit être déclarée avec aucun argument même si elle s'attend à recevoir les arguments spécifiés dans **CREATE TRIGGER** -- les arguments trigger sont passés via TG_ARGV, comme décrit plus loin. Quand une fonction PL/pgSQL est appelée en tant que trigger, plusieurs variables spéciales sont créées automatiquement dans le bloc de plus haut niveau. Ce sont :

NEW

Type de données RECORD ; variable contenant la nouvelle ligne de base de données pour les opérations

INSERT/UPDATE dans les triggers de niveau ligne. Cette variable est NULL dans un trigger de niveau instruction.

OLD

Type de données RECORD ; variable contenant l'ancienne ligne de base de données pour les opérations

UPDATE/DELETE dans les triggers de niveau ligne. Cette variable est NULL dans les triggers de niveau instruction.

TG_NAME

Type de données name ; variable qui contient le nom du trigger réellement lancé.

TG_WHEN

Type de données text ; une chaîne, soit BEFORE soit AFTER, selon la définition du trigger.

TG_LEVEL

Type de données text ; une chaîne, soit ROW soit STATEMENT, selon la définition du trigger.

TG_OP

Type de données text ; une chaîne, INSERT, UPDATE ou DELETE, indiquant pour quelle opération le trigger a été lancé.

TG_RELID

Type de données oid ; l'ID de l'objet de la table qui a causé le déclenchement du trigger.

TG_RELNAME

Type de données nom ; le nom de la table qui a causé le déclenchement.

TG_NARGS

Type de données integer ; le nombre d'arguments donnés à la procédure trigger dans l'instruction **CREATE TRIGGER**.

TG_ARGV[]

Type de donnée text ; les arguments de l'instruction **CREATE TRIGGER**. L'index débute à 0. Les indices invalides (inférieurs à 0 ou supérieurs ou égaux à tg_nargs) auront une valeur NULL.

Une fonction trigger doit renvoyer soit NULL soit une valeur record ayant exactement la structure de la table pour laquelle le trigger a été lancé. Les triggers de niveau ligne lancés BEFORE peuvent renvoyer NULL pour indiquer au gestionnaire de trigger de sauter le reste de l'opération pour cette ligne (les triggers suivants ne sont pas lancés, et les **INSERT/UPDATE/DELETE** ne se font pas pour cette ligne). Si une valeur non NULL est renvoyée alors l'opération se déroule avec cette valeur ligne. Renvoyer une valeur ligne différente de la valeur originale de NEW modifie la ligne qui sera insérée ou mise à jour (mais n'a pas d'effet sur le

cas **DELETE**). Pour modifier la ligne à stocker, il est possible de remplacer des valeurs seules directement dans NEW et de renvoyer NEW, ou de construire un nouveau record/ligne à renvoyer. La valeur de retour d'un trigger de niveau instruction BEFORE ou AFTER ou un trigger de niveau ligne AFTER est toujours ignoré ; il pourrait aussi bien être NULL. Néanmoins, tous les types de triggers peuvent toujours annuler l'opération complète en envoyant une erreur.

Application : Utiliser les déclencheurs (triggers) pour suivre la trace d'un agent prospecteur. Selon sa position, on peut savoir quelle est la station qui lui est la plus proche ou la zone où elle se trouve.

Exemple 1. Une procédure trigger PL/pgSQL

Cet exemple de trigger assure qu'à chaque moment où une ligne est insérée ou mise à jour dans la table, le nom de l'utilisateur courant et l'heure sont estampillés dans la ligne. Et cela vous assure qu'un nom d'employé est donné et que le salaire est une valeur positive.

```
CREATE TABLE emp (  
    nom_employe text,  
    salaire integer,  
    date_dermodif timestamp,  
    utilisateur_dermodif text  
);  
  
CREATE FUNCTION emp_stamp() RETURNS  
trigger AS $emp_stamp$  
BEGIN  
    -- Verifie que nom_employe et salary sont  
    donnés  
    IF NEW.nom_employe IS NULL THEN  
        RAISE EXCEPTION 'nom_employe ne  
    peut pas être NULL';  
    END IF;  
    IF NEW.salaire IS NULL THEN  
        RAISE EXCEPTION '% ne peut pas avoir  
    un salaire', NEW.nom_employe;  
    END IF;  
  
    -- Qui travaille pour nous si la personne doit  
    payer pour cela ?  
    IF NEW.salaire < 0 THEN  
        RAISE EXCEPTION '% ne peut pas avoir  
    un salaire négatif', NEW.nom_employe;  
    END IF;  
  
    -- Rappelons-nous qui a changé le salaire et  
    quand  
    NEW.date_dermodif := current_timestamp;  
    NEW.date_dermodif := current_user;  
    RETURN NEW;  
END;  
$emp_stamp$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER emp_stamp BEFORE
INSERT OR UPDATE ON emp
FOR EACH ROW EXECUTE PROCEDURE
emp_stamp();
```

Une autre façon de tracer les modifications sur une table implique la création d'une nouvelle table qui contient une ligne pour chaque insertion, mise à jour ou suppression qui survient. Cette approche peut être vue comme un audit des modifications sur une table. L'[Exemple 2](#) suivant montre un exemple d'une procédure d'audit par trigger en PL/pgSQL.

Exemple 2. Une procédure d'audit par trigger en PL/pgSQL

Cet exemple de trigger nous assure que toute insertion, modification ou suppression d'une ligne dans la table emp est enregistrée dans la table emp_audit. L'heure et le nom de l'utilisateur sont conservées dans la ligne avec le type d'opération réalisé.

```
CREATE TABLE emp (
    nom_employe    text NOT NULL,
    salaire        integer
);

CREATE TABLE emp_audit(
    operation      char(1) NOT NULL,
    tampon         timestamp NOT NULL,
    id_utilisateur text NOT NULL,
    nom_employe    text NOT NULL,
    salaire        integer
);

CREATE OR REPLACE FUNCTION
audit_employe() RETURNS TRIGGER AS
$emp_audit$
BEGIN
    --
    -- Ajoute une ligne dans emp_audit pour refléter
    l'opération réalisée
    -- sur emp,
    -- utilise la variable spéciale TG_OP pour cette
    opération.
    --
    IF (TG_OP = 'DELETE') THEN
        INSERT INTO emp_audit SELECT 'D', now(),
        user, OLD.*;
        RETURN OLD;
    ELSIF (TG_OP = 'UPDATE') THEN
        INSERT INTO emp_audit SELECT 'U', now(),
        user, NEW.*;
        RETURN NEW;
    ELSIF (TG_OP = 'INSERT') THEN
        INSERT INTO emp_audit SELECT 'I', now(),
        user, NEW.*;
```

```
RETURN NEW;
END IF;
RETURN NULL; -- le résultat est ignoré car il
s'agit d'un trigger AFTER
END;
$emp_audit$ language plpgsql;
```

```
CREATE TRIGGER emp_audit
AFTER INSERT OR UPDATE OR DELETE ON
emp
FOR EACH ROW EXECUTE PROCEDURE
audit_employe();
```

INDICATIONS

Enregistrez ce code dans un fichier texte de nom « Audit.sql ».

Lancez les commandes suivantes :

```
- createlang plpgsql mabase // ajouter le
langage de procédures PL/pgSQL dans la base
- psql -d mabase -f/repertoire/Audit.sql //
pour charger le script
- psql mabase
mabase# insert into emp
values('durant ',1000) ;
mabase# insert into emp
values('durant ',1000) ;
mabase# insert into emp
values('dupont ',2000) ;
mabase# update emp set salaire=3000 where
nom_employe ='durant ' ;
mabase# delete from emp where
nom_employe='durant' ;
mabase# select * from emp_audit ;
```

Une utilisation des triggers est le maintien d'une table résumée d'une autre table. Le résumé résultant peut être utilisé à la place de la table originale pour certaines requêtes -- souvent avec des temps d'exécution bien réduits. Cette technique est souvent utilisée pour les statistiques de données où les tables de données mesurées ou observées (appelées des tables de faits) peuvent être extrêmement grandes. L'[Exemple 3](#) suivant est un exemple d'une procédure trigger en PL/pgSQL maintenant une table résumée pour une table de faits dans un système de données (data warehouse).

Exemple 3. Une procédure trigger PL/pgSQL pour maintenir une table résumée

Le schéma détaillé ici est partiellement basé sur l'exemple du *Grocery Store* provenant de *The Data Warehouse Toolkit* par Ralph Kimball.

--

-- Tables principales - dimension du temps de ventes.

--

```
CREATE TABLE time_dimension (
  time_key          integer NOT NULL,
  day_of_week       integer NOT NULL,
  day_of_month      integer NOT NULL,
  month             integer NOT NULL,
  quarter           integer NOT NULL,
  year              integer NOT NULL
);
CREATE UNIQUE INDEX time_dimension_key
ON time_dimension(time_key);
```

```
CREATE TABLE sales_fact (
  time_key          integer NOT NULL,
  product_key       integer NOT NULL,
  store_key         integer NOT NULL,
  amount_sold       numeric(12,2) NOT
NULL,
  units_sold        integer NOT NULL,
  amount_cost       numeric(12,2) NOT
NULL
);
CREATE INDEX sales_fact_time ON
sales_fact(time_key);
```

--

-- Table résumé - ventes sur le temps.

--

```
CREATE TABLE sales_summary_bytime (
  time_key          integer NOT NULL,
  amount_sold       numeric(15,2) NOT
NULL,
  units_sold        numeric(12) NOT NULL,
  amount_cost       numeric(15,2) NOT
NULL
);
CREATE UNIQUE INDEX
sales_summary_bytime_key ON
sales_summary_bytime(time_key);
```

--

-- Fonction et trigger pour amender les colonnes résumées

-- pour un UPDATE, INSERT, DELETE.

--

```
CREATE OR REPLACE FUNCTION
maint_sales_summary_bytime() RETURNS
TRIGGER AS $maint_sales_summary_bytime$
DECLARE
```

```
  delta_time_key    integer;
  delta_amount_sold  numeric(15,2);
  delta_units_sold   numeric(12);
  delta_amount_cost  numeric(15,2);
BEGIN
```

-- Travaille sur l'ajout/la suppression de montant(s).

IF (TG_OP = 'DELETE') THEN

```
  delta_time_key = OLD.time_key;
  delta_amount_sold = -1 * OLD.amount_sold;
  delta_units_sold = -1 * OLD.units_sold;
  delta_amount_cost = -1 * OLD.amount_cost;
```

ELSIF (TG_OP = 'UPDATE') THEN

```
-- interdit les mises à jour qui modifient
time_key -
-- (probablement pas trop cher, car DELETE +
INSERT est la façon la plus
-- probable de réaliser les modifications).
IF ( OLD.time_key != NEW.time_key) THEN
  RAISE EXCEPTION 'Update of time_key : %
-> % not allowed', OLD.time_key, NEW.time_key;
END IF;
```

```
  delta_time_key = OLD.time_key;
  delta_amount_sold = NEW.amount_sold -
OLD.amount_sold;
  delta_units_sold = NEW.units_sold -
OLD.units_sold;
  delta_amount_cost = NEW.amount_cost -
OLD.amount_cost;
```

ELSIF (TG_OP = 'INSERT') THEN

```
  delta_time_key = NEW.time_key;
  delta_amount_sold = NEW.amount_sold;
  delta_units_sold = NEW.units_sold;
  delta_amount_cost = NEW.amount_cost;
```

END IF;

-- Insertion ou mise à jour de la ligne de résumé avec les nouvelles valeurs.

<<insert_update>>

LOOP

```
  UPDATE sales_summary_bytime
  SET amount_sold = amount_sold +
delta_amount_sold,
  units_sold = units_sold + delta_units_sold,
  amount_cost = amount_cost +
delta_amount_cost
  WHERE time_key = delta_time_key;
```

EXIT insert_update WHEN found;

BEGIN

```
  INSERT INTO sales_summary_bytime (
    time_key,
    amount_sold,
    units_sold,
    amount_cost)
  VALUES (
    delta_time_key,
```

```

delta_amount_sold,
delta_units_sold,
delta_amount_cost
);
EXIT insert_update;

EXCEPTION
WHEN UNIQUE_VIOLATION THEN
-- do nothing
END;
END LOOP insert_update;

RETURN NULL;

END;
$maint_sales_summary_bytime$ LANGUAGE
plpgsql;

```

CREATE TRIGGER

```

maint_sales_summary_bytime
AFTER INSERT OR UPDATE OR DELETE ON
sales_fact
FOR EACH ROW EXECUTE PROCEDURE
maint_sales_summary_bytime();

#INSERT INTO sales_fact
VALUES(1,1,1,10,3,15);
#INSERT INTO sales_fact
VALUES(1,2,1,20,5,35);
#INSERT INTO sales_fact
VALUES(2,2,1,40,15,135);
#INSERT INTO sales_fact
VALUES(2,3,1,10,1,13);
#SELECT * FROM sales_summary_bytime;
#DELETE FROM sales_fact WHERE
product_key = 1;
#SELECT * FROM sales_summary_bytime;
#UPDATE sales_fact SET units_sold = units_sold
* 2;
#SELECT * FROM sales_summary_bytime;

```

Exemple 4 d'application: Utiliser les déclencheurs (triggers) en PL/PGSQL de PostgreSQL pour suivre à la trace la personne 7 quand elle se déplace. Selon sa position, savoir quel est le bâtiment qui lui est le plus proche ou le bâtiment dans lequel elle se trouve?

La base comprend les tables postgis :

```

buildings
great_roads
mapserver_desc
parcs
personnes

```

```

rivers
small_roads

```

4.1. Fonction, table et trigger nécessaires

Le langage PL/PGSQL sert à tirer profit des procédures déclencheurs. Nous allons ici créer une table `suivi_de_la_personne_7` qui enregistrera à tout moment demandé enregistrera les informations suivantes:

- **utilisateur** correspondant au nom de l'utilisateur sous de PostgreSQL qui aura fait un update sur les données spatiales de la "personne 7";
- **date** correspondant à l'heure à laquelle a eu lieu le déplacement;
- **suivi** qui précisera si la personne est "dans" ou "proche" du bâtiment;
- **position** qui donnera la position de la personne.

Nous aurons aussi besoin d'une fonction `get_info_on_personne_7()` écrite en PL/PGSQL appelé par le déclencheur qui nous renverra le bâtiment qui contient la personne 7 et dans le cas contraire le premier bâtiment qui lui est le plus proche.

```

/*
Rappel pour effacer un trigger
drop trigger suivi_7_date on personnes;
*/

/*
Table qui contiendra le suivi de la
personne 7 lors de ses déplacements
avec effacement éventuel avant création
*/
select
drop_table_if_exists('suivi_de_la_personne_7',false);

create table suivi_de_la_personne_7(
utilisateur text,
date text,
suivi text,
position text
);
/*
Fonction appelée par le trigger
*/

```

```

create or replace function
get_info_on_personne_7() returns trigger as $$
declare
j record;
begin
    if (TG_OP = 'UPDATE') then
        SELECT into j data,distance,astext(the_geom)
from(
        SELECT a.data,Distance
(a.the_geom,b.the_geom),b.the_geom FROM
buildings a,personnes b
        WHERE b.data='personne 7'
        ORDER BY distance ASC limit 1)
as foo;
        if j.distance=0 then
            insert into suivi_de_la_personne_7 values
            (current_user,now(),'La personne 7 est
dans le batiment "'||j.data::text||'",j.astext::text);

RAISE NOTICE 'La personne 7 est dans le
batiment "%",j.data;
            else
            insert into suivi_de_la_personne_7 values
            (current_user,now(),'La personne 7 est proche
du batiment "'||j.data::text||'",j.astext::text);

RAISE NOTICE 'La personne 7 est proche du
batiment "%",j.data;

            end if;
        end if;
        return new;
end;

```

```

$$ language plpgsql;
/*

```

```

Création du trigger
*/
CREATE TRIGGER suivi_7_date AFTER
UPDATE ON personnes
EXECUTE PROCEDURE
get_info_on_personne_7();

```

4.2. Modifications de la position par la commande UPDATE et GeometryFromText

Procédons maintenant à quelques exemples de mise à jour des données spatiales de la personne 7:

```

madatabase=# update personnes set
the_geom=geometryfromtext('POINT(100.94
105.44)',-1) where data='personne 7';
INFO: La personne 7 est dans le batiment
'Bibliothèque Victor Hugo'
UPDATE 1

```

```

madatabase=# update personnes set
the_geom=geometryfromtext('POINT(100 70)',-1)
where data='personne 7';
INFO: La personne 7 est proche du batiment
'Office du Tourisme'
UPDATE 1
madatabase=# update personnes set
the_geom=geometryfromtext('POINT(120 66)',-1)
where data='personne 7';
INFO: La personne 7 est dans le batiment 'Office
du Tourisme'
UPDATE 1
madatabase=# update personnes set
the_geom=geometryfromtext('POINT(0 0)',-1)
where data='personne 7';
INFO: La personne 7 est proche du batiment
'Collège Arthur Rimbaud'
UPDATE 1
madatabase=# update personnes set
the_geom=geometryfromtext('POINT(60 87.5)',-1)
where data='personne 7';
INFO: La personne 7 est proche du batiment
'E.D.F'
UPDATE 1

```

4.3. Suivi des déplacements

La simple requête suivante nous renvoie les divers informations obtenues lors du déplacement de la personne 7

```

testgis=# select * from suivi_de_la_personne_7 ;
utilisateur |          date          | position
-----+-----
+-----+-----
postgres   | 2006-01-12 12:05:01.989762+01 | La
personne 7 est dans le batiment 'Bibliothèque
Victor Hugo' | POINT(100.94 105.44)
postgres   | 2006-01-12 12:05:20.824226+01 | La
personne 7 est proche du batiment 'Office du
Tourisme'   | POINT(100 70)
postgres   | 2006-01-12 12:05:41.106236+01 | La
personne 7 est dans le batiment 'Office du
Tourisme'   | POINT(120 66)
postgres   | 2006-01-12 12:06:05.024382+01 | La
personne 7 est proche du batiment 'Collège Arthur
Rimbaud'    | POINT(0 0)
postgres   | 2006-01-12 12:07:31.859971+01 | La
personne 7 est proche du batiment 'E.D.F'
| POINT(60 87.5)
(6 lignes)

```

C'est le genre de petite application qui peut par exemple servir lors de suivi par une application-

cliente (front-end) par le Web par exemple :
interface cartographique en SVG

4.4. Affichage avec MapServer

Afin d'obtenir un rendu adéquate, on va commencer par stocker les diverses positions dans une petite table que nous appellerons ici mouvement:

```
create table mouvement(id serial primary key);
select
addgeometrycolumn('mouvement','the_geom',-
1,'POINT',2);
insert into mouvement(the_geom)
values(geometryfromtext('POINT(100.94
105.44)',-1));
insert into mouvement(the_geom)
values(geometryfromtext('POINT(100 70)',-1));
insert into mouvement(the_geom)
values(geometryfromtext('POINT(120 66)',-1));
insert into mouvement(the_geom)
values(geometryfromtext('POINT(0 0)',-1));
insert into mouvement(the_geom)
values(geometryfromtext('POINT(60 87.5)',-1) );
```

Afin d'afficher les divers positions successives, on peut par exemple opter pour un layer comme suit

```
LAYER
  CONNECTION "user=david
dbname=madatabase host=localhost"
  CONNECTIONTYPE POSTGIS
  DATA "the_geom from mouvement"
  LABELITEM "id"
  METADATA
  END
  NAME "personnes"
  SIZEUNITS PIXELS
  STATUS DEFAULT
  TOLERANCE 0
  TOLERANCEUNITS PIXELS
  TYPE POINT
  UNITS METERS
  CLASS
  LABEL
    SIZE LARGE
    TYPE BITMAP
    BUFFER 0
    COLOR 0 0 255
    FORCE FALSE
    MINDISTANCE -1
    MINFEATURESIZE -1
    OFFSET 0 0
    PARTIALS TRUE
    POSITION CC
```

```
BACKGROUNDCOLOR 255 255 255
END
METADATA
END
STYLE
  ANGLE 360
  COLOR 255 0 0
  SIZE 2
  SYMBOL "circle"
END
END
END
```

On peut si on le souhaite joindre aussi les diverses positions successives entre elles

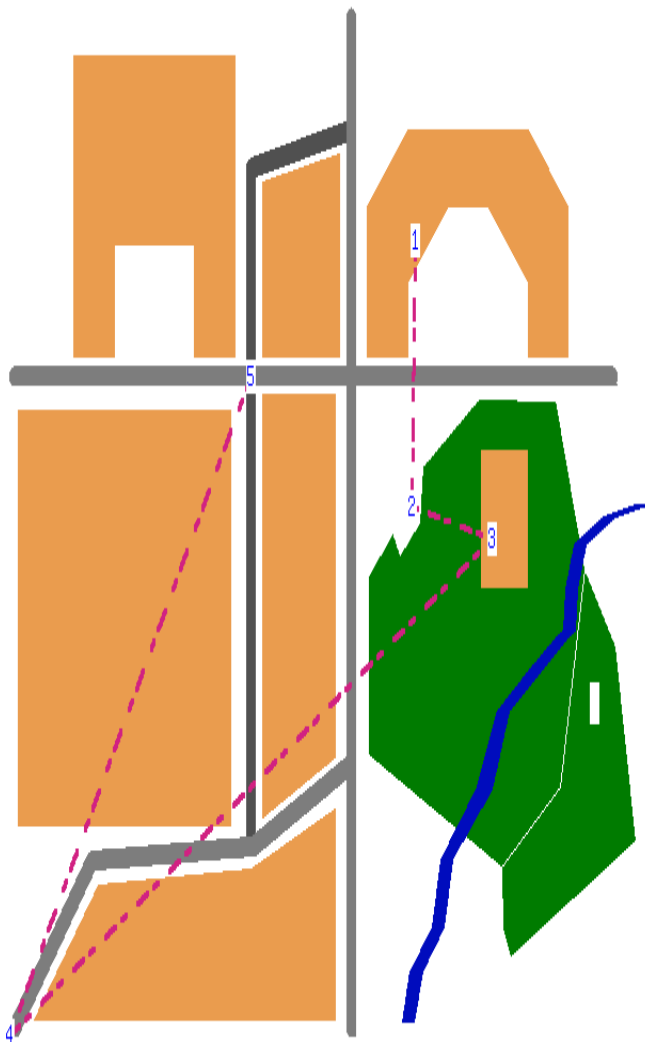
```
LAYER
  CONNECTION "user=david
dbname=madatabase host=localhost"
  CONNECTIONTYPE POSTGIS
  DATA "makeline from (select 1 as
id,makeline(the_geom) from mouvement) as foo
USING UNIQUE id USING SRID=-1"
  METADATA
  END
  NAME "requete_0"
  SIZEUNITS PIXELS
  STATUS ON
  TOLERANCE 0
  TOLERANCEUNITS PIXELS
  TYPE LINE
  UNITS METERS
  CLASS
  LABEL
    SIZE MEDIUM
    TYPE BITMAP
    BUFFER 0
    COLOR 22 8 3
    FORCE FALSE
    MINDISTANCE -1
    MINFEATURESIZE -1
    OFFSET 0 0
    PARTIALS TRUE
    POSITION CC
  END
  METADATA
  END
  STYLE
    ANGLE 360
    COLOR 209 33 130
    SIZE 3
    SYMBOL "dash"
  END
END
END
```

En ayant pris soin de définir quelque part dans la mapfile, le symbole "dash" pour une ligne discontinue

```
SYMBOL
NAME "dash"
TYPE ELLIPSE
POINTS 1 1 END
FILLED TRUE
STYLE 10 5 5 10 END
END
```

L'affichage sera alors le suivant

Figure 5.18. Déplacement successifs de la personne 7



NB: Il existe à ce jour quatre langages de procédures dans la distribution standard de **PostgreSQL™** : PL/pgSQL (PL/pgSQL - Langage de procédures SQL), PL/Tcl (PL/Tcl - Langage de procédures Tcl), PL/Perl (PL/Perl - Langage de procédures Perl) et PL/Python (PL/Python - Langage de procédures Python)